# Lecture 30: Practice Problems Suite (Questions)

## Supervised Machine Learning (Part 6) — Boosting & Advanced Ensembles

This problem set contains **exactly 3 case-based questions** derived strictly from Lecture 30 materials. Each question is structured around a practical scenario and explained simply.

---

## 🩺 Case Study 1: NovaGen Research Labs — Healthcare Risk Triage

### 🎯 Scenario & Background

NovaGen Research Labs collects health screening records for 9,549 patients ( `novagen_dataset.csv` ).

- **Features (22 predictors)**:
  - Continuous physiological indicators: `Age` , `BMI` , `Blood_Pressure` , `Cholesterol` , `Glucose_Level` , `Heart_Rate` , `Sleep_Hours` , `Exercise_Hours` , `Water_Intake` , `Stress_Level` .
  - Binary/categorical indicators: `Smoking` , `Alcohol` , `Diet` , `MentalHealth` , `PhysicalActivity` , `MedicalHistory` , `Allergies` , `Diet_Type__Vegan` , `Diet_Type__Vegetarian` , `Blood_Group_AB` , `Blood_Group_B` , `Blood_Group_O` .
- **Target**: `Target` (Binary: `1` = Clinical High Risk Patient, `0` = Low Risk Patient).
- **Clinical Rule**: Missing a high-risk patient (False Negative) can delay critical treatment. Therefore, **Recall (Sensitivity)** is the primary metric evaluated by doctors.

### 📝 Questions

1. **Baseline vs Ensemble Comparison**:
   - Split the data into 80% train and 20% test sets ( `random_state=42` ).
   - Standardize numerical features using `StandardScaler` .
   - Train the following 4 models:
     - `LogisticRegression(penalty='l2', solver='liblinear', max_iter=1000)`
     - `KNeighborsClassifier(n_neighbors=5)`
     - `RandomForestClassifier(n_estimators=200, random_state=42)`
     - `GradientBoostingClassifier(n_estimators=150, learning_rate=0.1, max_depth=3, random_state=42)`

- Compute and print the Recall score for each model on the test set.
- Explain in simple words: Why do the tree ensemble models (Random Forest and Gradient Boosting) achieve much higher recall (> 94%) than the linear model (~82%)?

2. **Soft Voting Classifier Implementation**:
   - Construct a `VotingClassifier` combining Logistic Regression, KNN, and a Decision Tree ( `max_depth=5` ):

```
voting_clf = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(solver='liblinear', max_iter=1000)),
        ('knn', KNeighborsClassifier(n_neighbors=5)),
        ('dt', DecisionTreeClassifier(max_depth=5, random_state=42))
    ],
    voting='soft'
)
```

   - Evaluate its test recall and accuracy.
   - Explain in simple terms: How does **Soft Voting** (averaging predicted probabilities) differ from **Hard Voting** (majority counting)? Why does soft voting usually perform better?
3. **Medical Deployment Decision**:
   - Review the final results table from `novagen.ipynb` :
     - Logistic Regression: 82.8% Recall
     - KNN: 88.3% Recall
     - Voting Classifier: 93.1% Recall
     - Gradient Boosting: 94.9% Recall
     - Random Forest: 95.8% Recall
   - If NovaGen must deploy a single production model, which model should be chosen and why?

---

## ⚙️ Case Study 2: Adaptive Sample Reweighting in AdaBoost

### 🎯 Scenario & Background

In high-precision manufacturing, an optical inspection system classifies microchips as Defective ( `y = +1` ) or Functional ( `y = -1` ). To minimize latency, we want to combine small, ultra-fast 1-split decision stumps ( `max_depth=1` ) using AdaBoost ( `adaboost.ipynb` ).

### 📝 Questions

1. **The Intuition of Sample Weights**:
   - Suppose we start with $N = 100$ microchip images. What is the initial weight $w_i^{(1)}$ assigned to each microchip in Round 1?
   - In Round 1, Decision Stump 1 misclassifies 10 microchips (weighted error $\epsilon_1 = 0.10$).
   - Explain in simple words:
     - How does AdaBoost update the weights of the 10 misclassified microchips?
     - How does it update the weights of the 90 correctly classified microchips?
     - Why does this force Decision Stump 2 in Round 2 to specialize in the hard microchips?
2. **Estimator Voting Power ($\alpha_m$)**:
   - The formula for the voting power of model $m$ is:

$$\alpha_m = \frac{1}{2} \ln \left( \frac{1 - \epsilon_m}{\epsilon_m} \right)$$

   - Calculate $\alpha_1$ for Stump 1 ($\epsilon_1 = 0.10$).
   - What would $\alpha_m$ be if a stump performs no better than random guessing ($\epsilon_m = 0.50$)?
   - What happens if a stump is nearly perfect ($\epsilon_m \to 0$)?
3. **Implementation & Boundary Progression**:
   - Train `AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1), n_estimators=50, learning_rate=0.5, random_state=42)` on synthetic classification data.
   - Explain the role of `learning_rate` (shrinkage parameter $\nu$): Why is a smaller learning rate (e.g. 0.5) usually combined with more estimators (e.g. 50 or 100) to prevent overfitting?

---

## 🏗️ Case Study 3: Meta-Learning Architectures — Stacking vs Blending

### 🎯 Scenario & Background

A financial risk assessment engine aims to predict default probability by combining diverse model architectures ( `ensemble_heterogenous.ipynb` ):

- Random Forest (Bagged non-linear trees)
- Gradient Boosting (Sequential error-correcting trees)
- Logistic Regression (Linear baseline)

## 📝 Questions

1. **Understanding Stacking in Simple Terms**:
   - Stacking is a two-layer meta-learning architecture:
     - **Layer 0 (Base Estimators)**: Multiple models make predictions.
     - **Layer 1 (Meta-Learner)**: A final estimator (e.g. Logistic Regression) learns which base model to trust for each type of input.
   - Explain why the base estimators should be **heterogeneous** (different algorithms) rather than all being identical trees.
2. **Stacking vs Blending — The Fundamental Difference**:
   - **Blending**: Splits the training data into two fixed sets (e.g., 70% Train, 30% Blend). Base models train on 70%, predict on 30%, and the meta-model trains only on the 30% predictions.
   - **Stacking**: Uses K-Fold Out-of-Fold (OOF) cross-validation so that 100% of the training data gets meta-predictions without data leakage.
   - What is the main trade-off between the two? (When would an engineer choose Blending over Stacking?)
3. **Scikit-Learn Implementation**:
   - Implement `StackingClassifier`:

```
from sklearn.ensemble import StackingClassifier

stacking_clf = StackingClassifier(
    estimators=[
        ('rf', RandomForestClassifier(n_estimators=100, random_state=42
        ('gb', GradientBoostingClassifier(n_estimators=100, random_stat
    ],
    final_estimator=LogisticRegression(),
    cv=5
)
```

   - Fit on training data and verify that meta-learning produces higher accuracy than either base model alone.